

7.1 Ordnungsrelationen

In dieser Aufgabe vergleichen wir auf 6 gleich bleibenden `std::vector`-Containern unterschiedliche Ordnungsrelationen. Dazu ist die Quelldatei `Exercise7_Ordering.cpp` gegeben, die bereits die notwendigen Funktionalitäten implementiert hat. Du musst in dieser Aufgabe nichts selbst programmieren.

Finde für jede der Teilaufgaben heraus, um welche Ordnungsrelation es sich handelt. Sollte es sich nicht um eine strenge Totalordnung handeln, gib einen Fall an, der die nächst-strengere Ordnung verletzt.

Tipp: Achte für die Substitution darauf, ob es eine offensichtliche Funktion geben kann, die für zwei äquivalente Vektoren unterschiedliche Ausgaben liefert.

- a) Führe das Programm aus und untersuche die Vergleiche in der Konsole.
- b) Ändere in Zeile 5 den definierten Buchstaben von `A` zu `B`, sodass in Zeile 5 stehen sollte: `#define B`. Führe das Programm aus und untersuche die Vergleiche in der Konsole.
- c) Ändere in Zeile 5 den definierten Buchstaben zu `C`. Führe das Programm aus und untersuche die Vergleiche in der Konsole.
- d) Ändere in Zeile 5 den definierten Buchstaben zu `D`. Schau dir an, was sich dadurch im Programmlauf ändert und überlege dir, was sich durch die Funktion `aeq()` im Vergleich zu Teilaufgabe C ändert und was nicht. Führe das Programm aus und untersuche die Vergleiche in der Konsole.

7.2 Array, Vector, List und String

Wir möchten vier unterschiedliche Möglichkeiten, um Zeichenketten zu speichern, genauer betrachten:

- `char[]`
- `std::vector<char>`
- `std::string`
- `std::list<char>`

Info:

In dieser Aufgabe geht es darum, die Unterschiede zwischen den Möglichkeiten zu untersuchen. Dabei sind sie in einfachen Anwendungsfällen meistens flexibel miteinander austauschbar und besitzen ähnliche Schnittstellen. Zum Beispiel lassen sich bei allen Möglichkeiten die Elemente mit einem gegebenen Iterator über `++` traversieren und mit `*` kann auch ein einzelnes Zeichen zugegriffen werden. Ihre Austauschbarkeit bietet den Vorteil, dass je nach Anwendungsgebiet die performanteste Lösung gewählt werden kann. Für die meisten Algorithmen auf Standard Containern existieren bereits effiziente Funktionen, nach denen zur Bearbeitung der Aufgabe gerne im Internet gesucht und die verwendet werden dürfen. Aber auch neben der Performanz gibt es unterschiedliche Eigenschaften zwischen ihnen:

- `Elem[]`: Kennt seine eigene Größe nicht. Besitzt die Funktionen `begin()` und `end()` sowie weitere nützliche Container-Funktionalitäten nicht. Range-Checks können nicht systematisch angewandt werden. Kann als Parameter an in C geschriebene Funktionen weitergegeben werden. Die Elemente liegen hintereinander im Speicher. Die Größe des Array ist zur Compile-Zeit festgesetzt. Vergleiche (`==` und `!=`) sowie Ausgabe (`<<`) finden auf dem Pointer auf das erste Element des Array und nicht auf den Elementen selbst statt.
- `std::vector<Elem>`: Ist flexibel einsetzbar, besitzt Iteratoren. Ermöglicht Indizierung. Listen-Operatoren wie `insert()` und `erase()` benötigen meistens das Verschieben der Elemente im Speicher (was bei großen und vielen Elementen sehr ineffizient sein kann). Range-Checks werden unterstützt. Die Elemente liegen hintereinander im Speicher. Ein `vector` kann erweitert werden (z.B. via `push_back()`). Vergleiche (`==`, `!=`, `<`, `<=`, `>` und `>=`) finden auf den Elementen selbst statt.
- `std::string`: Besitzt alle üblichen und nützlichen Operationen sowie zusätzlich spezifische Operationen zur Textmanipulation wie Konkatenation (`+` und `+=`). Die Elemente befinden sich hintereinander im Speicher. Ein `string` ist erweiterbar. Vergleiche (`==`, `!=`, `<`, `<=`, `>` und `>=`) finden auf den Elementen selbst statt.
- `std::list<Elem>`: Besitzt alle üblichen und nützlichen Operationen, außer Indizierung. Bei der Verwendung von `insert()` und `erase()` werden keine Elemente im Speicher

verschoben. Benötigt für jedes Element zusätzlichen Speicherplatz (für die Zeiger auf das vorherige und nächste Element). Eine `list` ist erweiterbar. Vergleiche (`==`, `!=`, `<`, `<=`, `>` und `>=`) finden auf den Elementen selbst statt.

- a) Definiere für die vier Möglichkeiten eine Instanz mit dem Wert `"Hello"`, übergib sie einer Funktion als Parameter, gib die Anzahl der Zeichen in dem übergebenen Parameter aus, vergleiche ihn in der Funktion mit dem Wert `"Hello"` (zur Überprüfung, ob wirklich `"Hello"` übergeben wurde), und vergleiche ihn mit dem Wert `"Howdy"`, um zu schauen, was in einem Lexikon zuerst kommen würde. Kopiere den Funktionsparameter in eine andere Variable desselben Typs.
- b) Erstelle ein `int[]`, `std::vector<int>` und `std::list<int>` mit dem Wert `{1,2,3,4,5}` und ein `std::string` mit dem Wert `"12345"`. Für jede der vier Instanzen: Übergib sie einer Funktion als Parameter, gib seine Länge aus, kopiere ihn in eine andere Variable desselben Typs. Dreh die Reihenfolge der Elemente in der Kopie um und gib sie in der Konsole aus. Durchsuche den ursprünglichen Parameter und seine Kopie nach dem Element `4` (wenn möglich mit der Funktion `std::find()`) und gib die Position des Elements aus.

7.3 Experiment: Performance von Vector und List

In dieser Aufgabe sollen die Performance Unterschiede zwischen `std::vector` und `std::list` ein wenig genauer betrachtet werden. Die Quelldatei `Exercise7_Performance.cpp` liefert dir dazu das Grundgerüst, in dem du alle Teilaufgaben lösen kannst.

- a) Die Funktion `exerciseA()` erhält einen leeren `std::vector<int>` als Eingabeparameter. Erzeuge $n \in \mathbb{N}$ zufällige Integerwerte im Intervall $[0, N)$. Füge jeden Zufallswert direkt nach der Erzeugung dem `vector` hinzu (dieser wächst also jedes Mal um ein Element). Sorge dafür, dass der `vector` sortiert ist, indem du den Zufallswert direkt so hinzufügst, dass dieser nach jedem Wert eingefügt wird, der kleiner oder gleich dem erzeugten Zufallswert ist, und vor jedem Wert, der größer oder gleich ist. Mithilfe der Funktion `elapsed_seconds.count()` wird ausgegeben, wie lange die Erzeugung und das Hinzufügen der Werte gedauert hat.
- b) Die Funktion `exerciseB()` erhält einen leeren `std::list<int>` als Eingabeparameter. Wiederhole das Experiment aus Teilaufgabe a für die Liste. Vergleiche die Varianten mit `list` und `vector` für unterschiedliche $n \in \mathbb{N}$ und Zufalls-Seeds. Kommentiere bei großen n die Konsolenausgabe für die einzelnen Zeiten und den sortierten Container aus. Was fällt dir auf? Versuche eine Erklärung für deine Erkenntnisse zu finden.
- c) Ändern sich die Erkenntnisse, wenn bei dem `std::vector` vor der Übergabe an die Funktion `exerciseA()` mit `.reserve(n)` ausreichend Speicher für die Elemente allokiert wird?
- d) Kopiere deinen selbst geschriebenen Code aus der Funktion `exerciseA()` an die entsprechende Stelle in `exerciseD()`. In `exerciseD()` wird bereits nach dem Hinzufügen eines neuen Wertes zusätzlich allokiert Speicher wieder freigegeben. Die Freigabe des Speichers fließt dabei nicht in die Zeiterfassung hinein. Wie würdest du das Zeitverhalten der Funktion im Vergleich zu den Experimenten der vorherigen Teilaufgaben prognostizieren? Überprüfe deine Prognose, indem du die Funktion entsprechend aufrufst.
- e) *Zusatzaufgabe:* Die Funktion `exerciseE()` erhält nun eine leere `std::multiset<int>` als Eingabeparameter. Wiederhole das Experiment aus Teilaufgabe a für die Menge und versuche das Zeitverhalten im Vergleich mit den vorherigen Experimenten zu erklären.

7.4 Bibliothek 2.0

Die Bibliotheks-Software aus Ausgabe 4.2 soll erweitert werden. Das UML-Diagramm aus Abbildung 1 gibt dir einen Überblick über die Software nach allen Erweiterungen. Die Teilaufgaben werden dich durch die einzelnen Erweiterungen hindurchführen. Beachte, dass ein UML-Diagramm die Klassenstruktur vereinfacht und unabhängig von der verwendeten Programmiersprache darstellen soll. Das bedeutet, dass zum Beispiel nicht angegeben werden muss, wenn es sich bei einem Parameter um einen Pointer handelt.

Hinweis: Wenn du zur Bearbeitung dieser Aufgabe nicht auf deiner eigenen Umsetzung von Aufgabe 4.2 aufbauen möchtest, darfst du gerne den Lösungsvorschlag von Aufgabe 4.2 verwenden.

- a) Füge der Bibliotheks-Software die Klasse `Patron` hinzu. Sie besitzt einen Nutzernamen, die Nummer der Bibliothekskarte und mögliche Leihschulden. Erstelle *Getter*-Funktionen, um auf die Daten zuzugreifen, sowie eine *Setter*-Funktion für die Leihschulden. Füge der Klasse eine Helfer-Funktion hinzu, die einen booleschen Wert dafür zurück gibt, ob der Nutzer Schulden hat oder nicht.
- b) Erstelle die Klasse `Library`, die einen `std::vector` für Instanzen der Klasse `Patron` und ein `std::set` für Instanzen der Klasse `Publication` besitzt. Die Publikationen sollen nach ihrer Identifikation sortiert sein. Die Klasse soll Funktionen beinhalten, die es ermöglichen, Publikationen und Nutzer der Bibliothek hinzuzufügen. Zusätzlich sollen Funktionen existieren, mit denen nach Nutzern und Publikationen gesucht werden können und die Information (bool) zurückgibt, ob das gesuchte Element in der Bibliotheks-Software existiert.
- c) Erstelle ein `struct` namens `Transaction`, welches eine Publikation, einen Nutzer und einen Zeitpunkt* speichert. Die Klasse `Library` soll einen `std::vector` aus Transaktionen besitzen. Füge der Klasse `Library` eine Funktion hinzu, mit der Bücher ausgeliehen werden können. Überprüfe in der Funktion, ob sowohl der ausleihende Nutzer als auch die auszuleihende Publikation in der Software vorhanden sind und gib einen Fehler auf der Konsole aus, wenn dies nicht der Fall ist. Sollte der Nutzer Schulden besitzen, gib ebenfalls einen Fehler aus. Besitzt der Nutzer keine Schulden und ist die auszuleihende Publikation momentan verfügbar, erstelle eine entsprechende Transaktion mit der aktuellen Zeit als Zeitpunkt. Rufe außerdem die Funktion `borrow()` der Publikation auf.

***Information zu Zeiten in C++:** Informiere dich über die `std::chrono` Funktionalitäten aus der Bibliothek `<chrono>`, um den aktuellen Zeitpunkt abzuspeichern, mit anderen Zeiten zu vergleichen und als Datum mit Uhrzeit auszugeben.

- d) *Zusatzaufgabe:* Füge der Klasse `Library` eine Funktion `giveBack(const Patron*, Publication*)` (Mit beliebiger Pointer-Realisierung) und eine Funktion `checkFees()` hinzu.
 - Die Funktion `Library::giveBack()` soll in der Liste der Transaktionen überprüfen, ob die angegebene Publikation vom angegebenen Nutzer ausgeliehen ist. Ist das der Fall, soll die Transaktion aus dem Vektor gelöscht werden und die Funktion `Publication::giveBack()`

der Publikation aufgerufen werden. Ist das nicht der Fall, so soll eine Fehlermeldung in der Konsole ausgegeben werden.

- Die Funktion `checkFees()` soll für alle Transaktionen überprüfen, ob die Publikationen bereits so lange ausgeliehen sind, dass der Nutzer Leihschulden erhält. Wähle die Zeit dafür so, dass sie sich gut testen lässt.

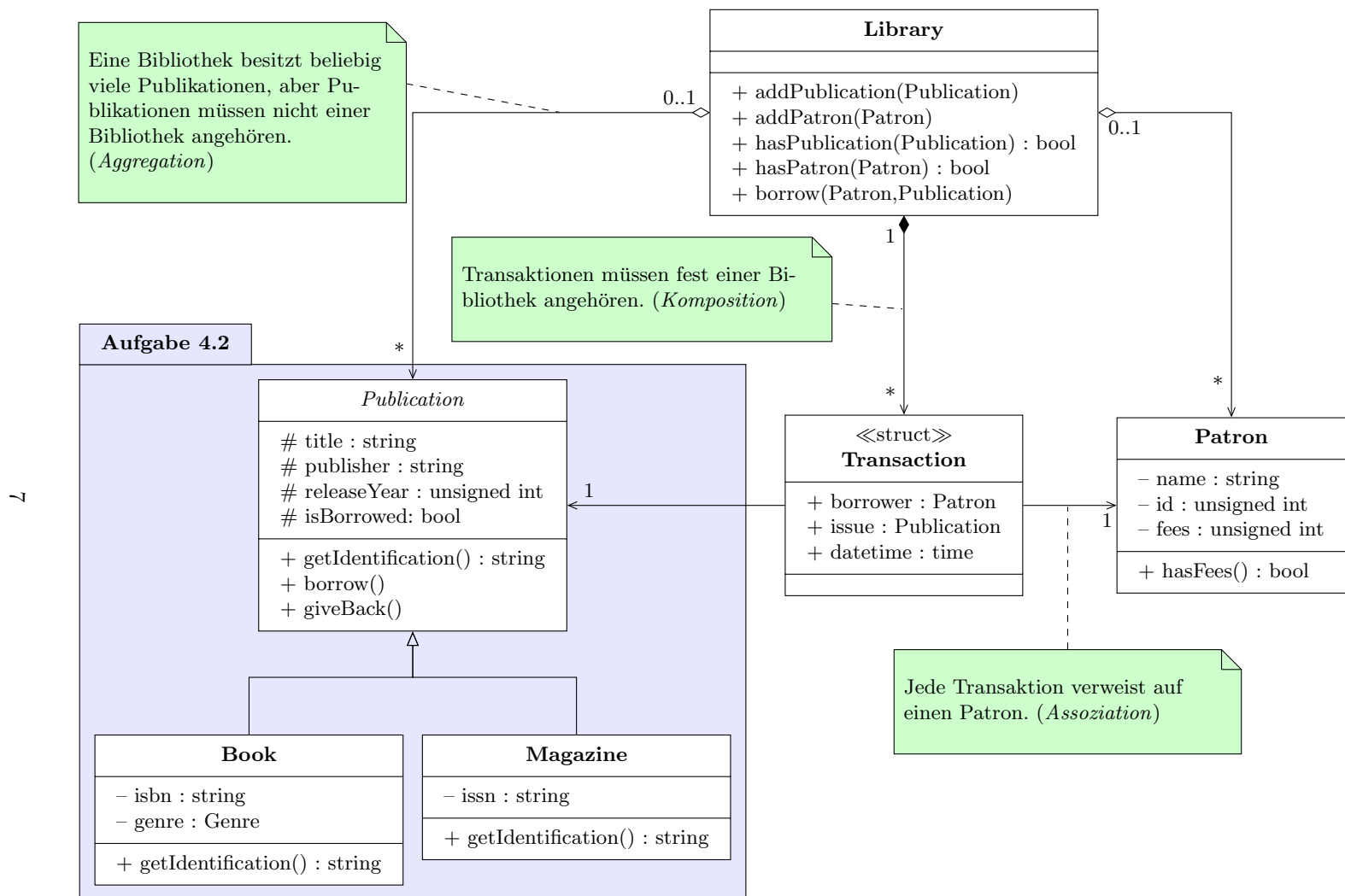


Abbildung 1: Bibliothek-Software